

5.4 Feature Analysis and Visualization

5.4.1 UMAP Embedding and Clustering

We performed dimensionality reduction on the normalized SAE decoder weights using UMAP (parameters: metric="euclidean", neighbors=15, min dist=0.1). Clustering was performed using HDBSCAN [25] (min cluster size=5, min samples=3) for visualization in the InterPLM interface.

5.4.2 Sequential and Structural Feature Analysis

We characterized features' sequential and structural properties using the following procedure:

1. Identified high-activation regions (>0.6) in proteins with available AlphaFold structures
2. For each protein's highest-activation residue, calculated:
 - Sequential clustering: Mean activation within ± 2 positions in sequence
 - Structural clustering: Mean activation of residues within 6Å in 3D space
3. Generated null distributions through averaging 5 random permutations per protein
4. Generated null distributions through averaging 5 random permutations per protein
5. Assessed clustering significance using paired t-tests and Cohen's d effect sizes across 100 proteins per feature

The 100 proteins per feature were randomly sampled from our Swiss-Prot dataset. Features with < 25 examples meeting this criteria were excluded from this analysis. For structural feature identification in InterPLM, we considered only proteins with Bonferroni-corrected structural p-values < 0.05 , with features colored based on the ratio of structural to sequential effect sizes.

5.5 Steering Experiments

Following the approach described in [20] we decomposed ESM embeddings into SAE reconstruction predictions and error terms. For sequence steering, we (1) Extracted embeddings at the specified layer (2) Calculated SAE reconstructions and error terms (3) Modified the reconstruction by clamping specified features to desired values (4) Combined modified reconstructions with error terms (5) Allowed normal model processing to continue (6) Extracted logits and calculated softmax probabilities for comparison across steering conditions. Steering experiments all conducted using NNsight [26].

Acknowledgements

Special thanks to Tom McGrath, Alex Tamkin, Nicholas Joseph, and Zou lab members for helpful discussions and feedback. E.S. is supported by NSF GRFP (DGE-2146755) and J.Z. is supported by funding from the CZ Biohub.

A Additional background on Protein Language Models

Protein language models (PLMs) adapt techniques from natural language processing to learn representations of protein sequences that capture both structural and functional properties. These models typically use transformer architectures, treating amino acids as discrete tokens. While natural language modeling typically uses an autoregressive training method, many modern PLMs employ masked language modeling objectives similar to BERT, as protein sequences fundamentally exhibit bidirectional dependencies - the folding and function of any given amino acid depends on both N-terminal and C-terminal context. These models learn impressively rich protein representations through self-supervised training on large sequence databases without requiring structural annotations. The learned embeddings capture hierarchical information ranging from local physicochemical properties to global architectural features. Notably, these representations have proven crucial for protein structure prediction - they serve as the input embeddings for dedicated folding models like ESMFold, and even AlphaFold, while not explicitly a language model, dedicates most of the computational resources in a given prediction to learning protein representations from multiple sequence alignments in a conceptually similar manner. Many efforts have demonstrated that the representations learned through masked language modeling alone contain remarkable structural and functional information, enabling state-of-the-art performance on tasks ranging from structure prediction to protein engineering. This success appears to stem from the models' ability to capture the underlying patterns in evolutionary sequence data that reflect physical and biological constraints.

B Extended SAE Details and Analysis

B.1 Additional background

SAEs transform the latent vector for a single amino into a new vector with increased size and sparsity. When a specific position in this vector has a non-zero value, that corresponds to the presence of a specific pattern in the amino acid’s neuron embedding. Ideally these model patterns correspond to human interpretable features that we can understand based on patterns by which the feature activates and the impact it has on the model when activated. Specifically, when we perform analysis on feature activation levels, these are using the $f_i(x)$ values, while dictionary value analysis uses the learned weights in W_{d_i} as visualized in 8.

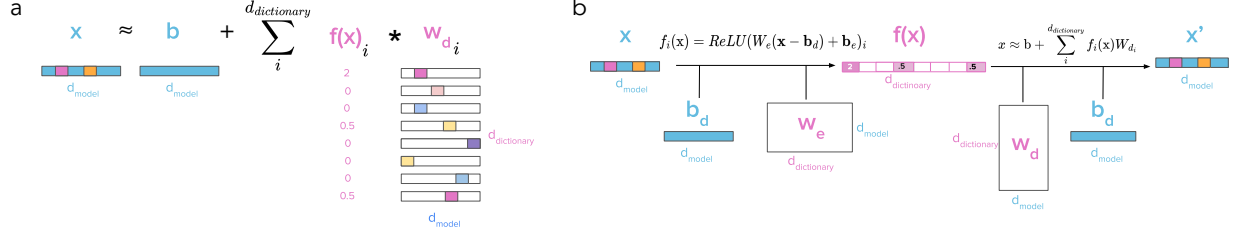


Figure 8: Overview of SAE decomposition and training.

(a) Decomposition of embedding vector into weighted sum of dictionary elements. (b) Architecture for the SAE

While the SAEs trained in this paper use the architecture above, there are many other alternate SAE methods that vary weight initializations, nonlinearities, loss function, and other training details in aims to simultaneously increase sparsity and increase the reconstruction accuracy [27][28][29].

B.1.1 SAE metrics

While the goal of SAE training is to learn features that are maximally interpretable and accurate, these qualities are challenging to explicitly optimize for so during training we optimize for sparsity, hoping that more sparse features are more interpretable, and reconstruction quality.

Specifically, during training we calculate our loss as a weighted sum of an L1 norm calculating the absolute sum of all feature values, and mean squared error calculating how close the reconstructed x' is to the original x . Then, when training SAEs or comparing different implementations, people evaluate L_0 , the average number of nonzero elements in the latent representation per token and Percent Loss Recovered, the percent of the original cross entropy of the model that is achieved when the model’s embeddings are replaced by reconstructions. This last metric measures the amount ‘recovered’ by comparing the cross entropy of the model using reconstructed embeddings ($CE_{\text{Reconstruction}}$) to the original cross entropy (CE_{Original}) and the cross entropy when the specified embedding layer is instead replaced with all zeros (CE_{Zero}) but all later layers remain identical.

Metrics described are calculated per these equations:

$$L_1(f(x)) = \sum_{i=1}^{d_{\text{dictionary}}} |f_i(x)|$$

$$MSE(x, x') = \frac{1}{d_{\text{dictionary}}} \sum_{i=1}^{d_{\text{model}}} (x_i - x'_i)^2$$

$$L_0(f(x)) = \sum_{i=1}^{d_{\text{dictionary}}} \mathbb{1}(f_i(x) > 0)$$

$$\% \text{ Loss Recovered} = \left(1 - \frac{CE_{\text{Reconstruction}} - CE_{\text{Original}}}{CE_{\text{Zero}} - CE_{\text{Original}}} \right) * 100$$

While we do our final model selection based on Swiss-Prot concept associations as this more closely targets *biological interpretability* than pure sparsity, initial experiments optimized final hyperparameter ranges with these metrics. Final hyperparameters along with L_0 and % Loss Recovered for our 6 SAEs are:

Layer	Learning Rate	L1	% Loss Recovered
L1	1.0e-7	0.1	99.60921
L2	1.0e-7	0.08	99.31827
L3	1.0e-7	0.1	99.02078
L4	1.0e-7	0.1	98.39785
L5	1.0e-7	0.1	99.32478
L6	1.0e-7	0.09	100

Table 1: Layer-wise learning parameters and SAE performance metrics

C Swiss-Prot Concept Associations

C.1 Swiss-Prot Metadata Categories

These tables contain all of the metadata used for quantitative feature-concept associations and LLM feature descriptions and validation, organized into groups with similar themes. The last two columns specify whether each field was used in quantitative concept analysis, LLM descriptions, or both.

C.2 Swiss-Prot Metadata Categories

These tables contain all of the metadata used for quantitative feature-concept associations and LLM feature descriptions and validation, organized into groups with similar themes. The last two columns specify whether each field was used in quantitative concept analysis, LLM descriptions, or both.

C.2.1 Basic Identification Fields

Field Name	Full Name	Description	Quant.	LLM
accession	Accession Number	Unique identifier for the protein entry in UniProt	N	Y
id	UniProt ID	Short mnemonic name for the protein	N	Y
protein_name	Protein Name	Full recommended name of the protein	N	Y
gene_names	Gene Names	Names of the genes encoding the protein	N	Y
sequence	Protein Sequence	Complete amino acid sequence of the protein	N	Y
organism_name	Organism	Scientific name of the organism the protein is from	N	Y
length	Sequence Length	Total number of amino acids in the protein	N	Y

C.2.2 Structural Features

Field Name	Full Name	Description	Quant.	LLM
ft_act_site	Active Sites	Specific amino acids directly involved in the protein’s chemical reaction	Y	Y
ft_binding	Binding Sites	Regions where the protein interacts with other molecules	Y	Y
ft_disulfid	Disulfide Bonds	Covalent bonds between sulfur atoms that stabilize protein structure	Y	Y
ft_helix	Helical Regions	Areas where protein forms alpha-helical structures	Y	Y
ft_turn	Turns	Regions where protein chain changes direction	Y	Y
ft_strand	Beta Strands	Regions forming sheet-like structural elements	Y	Y
ft_coiled	Coiled Coil Regions	Areas where multiple helices intertwine	Y	Y
ft_non_std	Non-standard Residues	Non-standard amino acids in the protein	N	Y
ft_transmem	Transmembrane Regions	Regions that span cellular membranes	N	Y
ft_intramem	Intramembrane Regions	Regions located within membranes	N	Y